AI for Lexical & Syntax Assistance

Name: Neelam Jashwanth

Roll Number: 24CSB0A49

Phase: Phase 1 (Exploration & Analysis)

WEEK 1 REPORT: Problem Definition & Context Understanding

1. **Problem Statement:** In the domain of software development, particularly for novice programmers and students, debugging is a significant bottleneck. Traditional compilers (such as GCC, Clang, or Javac) are designed primarily to detect errors and halt execution. However, the error messages provided by these compilers are often cryptic, technical, and difficult to interpret.

    - **Example:** A missing semicolon might generate a cascading list of errors that do not point to the actual source of the problem.

    - **Limitation:** While standard compilers identify *that* an error exists, they rarely provide actionable intelligence on *how* to fix it automatically. This project addresses the need for an intelligent system that acts as a "Grammarly for Code," detecting lexical and syntactic errors and providing automated, context-aware corrections.

2. **Project Objectives:** The primary goal of this project is to design and implement an AI-assisted compiler front-end. The specific objectives are:

    - **To Develop an Intelligent Tokenizer:** To build a custom tokenizer capable of handling malformed code inputs that would typically crash a standard lexer.

- **To Implement AI-Driven Error Correction:** To utilize Deep Learning models (specifically Sequence-to-Sequence models like LSTMs or Transformers) to predict the correct code sequence from erroneous input.

- **To Enhance Developer Efficiency:** To reduce the time spent on debugging syntax errors by offering one-click fixes for common mistakes (e.g., missing delimiters, typos in keywords, unmatched parentheses).

- **To Ensure Security:** To analyze whether syntax corrections introduce any logic that could lead to vulnerabilities.

3. Scope of the Project

- **Input Domain:** The system will focus on **C Language** and **Python** code snippets.
- Error Types:

  - *Lexical Errors:* Misspelled keywords (e.g., init instead of int, whille instead of while).
  - *Syntactic Errors:* Punctuation errors (missing ;, {, }), unbalanced parentheses, and malformed control structures.

- **Exclusions:** The tool will not address complex semantic errors (like logical flaws or algorithmic correctness) in this version.

4. Methodology Overview The project will follow a machine learning pipeline approach:

I. **Data Collection:** Gathering datasets of correct and buggy C
II. programs.**Preprocessing:** Tokenizing code and injecting synthetic errors for training.
III. **Model Training:** Training an LSTM/Transformer model to map "Buggy Code" to "Clean Code".
IV. **Integration:** Interfacing the trained model with a compiler front-end script.